

BUSN 41210 — USING AI TOOLS WELL

Financial Analytics · Autumn 2026 · read this before Homework 1

The policy is simple: you may use ChatGPT, Claude, Copilot or anything similar on every assignment in this course, and you do not need to cite it. This handout is not about permission. It is about getting more out of these tools than a classmate who pastes the question in and hands back whatever comes out.

The standard you are held to is unchanged: you must be able to defend every line you submit. “The AI wrote it” is not an answer to “why is there a `shuffle=True` there?” — and that particular question will cost you marks, for reasons this handout gets to.

1. What it is genuinely good at

Use it freely, and without guilt, for all of these:

- **Syntax you have forgotten.** “How do I group by two columns and take the median in pandas?”
- **Plotting.** Getting a legend where you want it, two panels the same height, a log axis. This is the single biggest time-saver in the course.
- **Reading an error message.** Paste the whole traceback. It is very good at this, and a traceback is exactly the kind of thing that stops a beginner for an hour and an AI for four seconds.
- **Explaining code you were given.** Paste a cell from a lecture notebook and ask what each line does. This is a legitimate and excellent way to study.
- **Explaining the statistics.** See §5 — this is underused, and it is where students new to the subject get the most out of it.
- **Translating.** R to Python, a formula on a slide into code, a loop into a vectorised expression.

2. What it is unreliable at

Not “never use it for these” — *always check it* for these:

- **Choosing the method.** It will confidently propose a random forest for a problem that needs a time-ordered split, or a *t*-test where the observations are not independent.
- **Anything about your actual data.** It cannot see your file. It will guess column names, assume a date is a date, and invent a plausible number of rows. It does not know your interest rate column is stored as the string ' 10.65%' — the Lecture 6 credit data really is like that.
- **Validation design.** This is the big one, and §4 is about it.
- **Interpreting a result.** It will tell you an R^2 of 0.72 is good. Whether 0.72 is good depends on the benchmark, and it does not know your benchmark.
- **Current library versions.** It has read a great deal of code written for pandas 1.x, and this course runs pandas 3. See the table in §3 — every one of those is verified on our environment.

3. Five things it writes that will simply fail here

Measured on the course environment (Python 3.14, pandas 3.0.5, scikit-learn 1.9.0):

what an older model tends to write	what happens
<code>df.append(row, ignore_index=True)</code>	AttributeError — removed in pandas 2.0; use <code>pd.concat</code>
<code>s.fillna(method='ffill')</code>	TypeError — use <code>s.fffll()</code>
<code>s.iteritems()</code>	AttributeError — use <code>s.items()</code>
<code>LassoCV(normalize=True)</code>	TypeError — the argument is gone; put a <code>StandardScaler</code> in a <code>Pipeline</code>

what an older model tends to write	what happens
<code>sns.set(...)</code> / very old matplotlib kwargs	assorted deprecation noise

None of these is dangerous — they crash, you notice, you fix them. **The dangerous ones are in §4, because they do not crash.**

4. Four things it writes that run fine and are wrong

These produce a number. The number is wrong. Nothing warns you.

(a) **`train_test_split(X, y)` on time-ordered data.** Its default is `shuffle=True` — verified. On a macro panel or a return series that puts 2021 in the training set and scores you on 2015. Your out-of-sample R^2 will look wonderful and mean nothing. *Graded as fatal in every assignment.*

(b) **Fitting a scaler, an imputer or a PCA before the split.**

```
X_scaled = StandardScaler().fit_transform(X)           # <- learned from ALL the data
X_tr, X_te = X_scaled[:n], X_scaled[n:]                # too late
```

The right shape is a `Pipeline` fitted after the split, so the transformation only ever sees training rows. Lecture 7 measures what this costs; Lecture 6 shows the version of it that put 27,178 loans on both sides of a split. *Also graded as fatal.*

(c) **`LogisticRegression()` described as “plain” or “unregularized” logistic regression.** It is not. Verified on our environment: the default applies an L2 penalty at `C=1.0`, which shrank the coefficients in a five-variable test by about **40%**. scikit-learn’s own docstring says “*Note that regularization is applied by default.*” If you want the textbook estimator, you have to ask for it.

(d) **`r2_score(y_test, y_pred)` reported as “the out-of-sample R^2 ”.** `r2_score` measures against the **test-set mean**, which nobody could have known in advance. This course scores against a benchmark fixed before the split. The two differ, and the difference is not always small.

5. Using it to learn the statistics, not just the syntax

If you have not done much statistics before, this is the most valuable section here. These prompts work well — they ask it to *teach*, which it is good at, rather than to *decide*, which it is not.

- “*Explain what a standard error is to someone who has never taken statistics, using a concrete example with numbers.*”
- “*I fitted this regression and got these coefficients. Walk me through what each number means, in plain English, one at a time.*”
- “*What does it mean that my out-of-sample R^2 is negative? Is that possible? What would cause it?*”
- “*Here is my code and here is what my professor said about leakage. Which lines could be leaking, and why?*”
- “*Give me three ways this result could be wrong that I have not thought of.*”
- “*Ask me questions until you understand what I am trying to do, then suggest an approach.*” — this one is surprisingly effective; it stops the tool from guessing.

And the one to use before you submit anything:

“*Here is my notebook. Pretend you are a sceptical referee. What would you attack first?*”

6. A workflow that works

1. **Describe your data before you ask for code.** Paste `df.head()` and `df.dtypes`. Most bad answers come from the tool guessing what your data looks like.
2. **Ask for one step at a time.** A 60-line block you cannot read is worse than five lines you can.

3. **Read it before you run it.** Every line. If you do not know what a line does, ask — that is a free lesson, and you need it for the defence anyway.
4. **Run it and check it against something you already know.** Does the number of rows match? Does the coefficient have the sign the lecture said it should? Does the in-sample fit exceed the out-of-sample fit?
5. **Verify the thing that matters.** Whatever the headline number is, ask yourself what would have to be true for it to be believable, then check that thing.
6. **When it is confidently wrong, say so and give it the evidence.** “That raised `TypeError: ...` on pandas 3” will usually get you a correct answer on the second try.

7. Where this shows up in the course

- **The two §4 mistakes that involve your split — (a) and (b) — are graded as fatal in every assignment**, and AI-written code makes them more often than hand-written code does, because the common patterns on the internet are written for cross-sections.
- **Every write-up should be able to say what you tried and rejected** — which is very hard to do honestly if you did not read your own code.
- **Homework is graded on the write-up as well as the answer.** A correct number with an explanation you cannot give is worth less than a wrong number you understand.

The one-sentence version. Use it for syntax, plots, error messages and explanations without hesitation; check everything it says about *your data*, *your split* and *whether a number is good*; and never submit a line you could not defend if asked.